Лабораторная работа 3

Лабораторная работа: Форма обратной связи с валидацией и состояниями в React

Цель работы

Научиться **управлять состоянием** формы в React с помощью хука useState, реализовать клиентскую валидацию полей и стилизовать различные состояния полей (**покой, фокус, ошибка, успех**).

Теоретическая справка

Хук useState — это функция React, которая позволяет функциональному компоненту хранить и обновлять локальное состояние. Возвращает массив из двух элементов: текущего значения и функции-сеттера.

jsx

const [value, setValue] = useState(initialValue);

Управляемые компоненты — поля формы, значение которых контролируется React-состоянием. Каждый ввод символа обновляет состояние, а состояние определяет отображаемое значение.

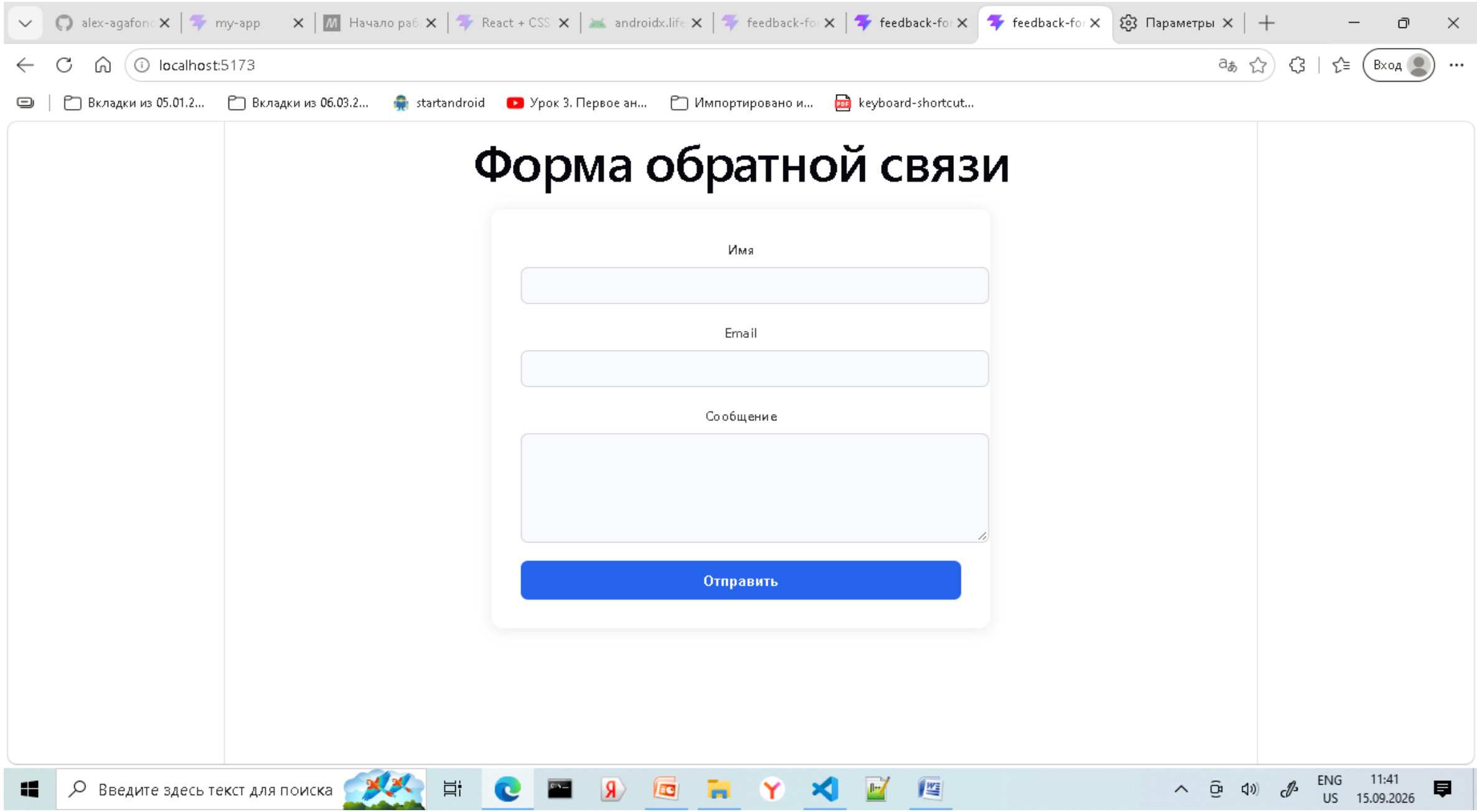
Валидация — проверка данных перед отправкой. В React обычно реализуется в двух моментах:

при изменении поля (on-the-fly);

при отправке формы (on submit).

Состояния полей — визуальные подсказки пользователю:

покой (idle), фокус (focus), ошибка (error), успех (valid).


alex-agafonc
my-app
Начало раб
React + CSS
androidx.life
feedback-for
feedback-for
feedback-for
Параметры
localhost:5173
Вход
Вкладки из 05.01.2...
Вкладки из 06.03.2...
startandroid
Урок 3. Первое ан...
Импортировано и...
keyboard-shortcut...
Форма обратной связи
Имя
Email
Сообщение
Отправить
Введите здесь текст для поиска
ENG
US
11:41
15.09.2026

Подготовка проекта

Создайте новый проект через Vite:

```
npm create vite@latest feedback-form -- --template react
cd feedback-form
npm install
npm run dev
```

Структура, с которой будем работать:

```
src/
 ├── App.jsx
 ├── App.css
 ├── components/
 │    ├── FeedbackForm.jsx
 │    └── FeedbackForm.css
```

Пошаговое выполнение

Шаг 1. Каркас компонента формы

Создайте файл src/components/FeedbackForm.jsx. Начнём с базовой структуры — полей «Имя», «Email», «Сообщение» и кнопки отправки.

```
// src/components/FeedbackForm.jsx

import { useState } from "react";
import "./FeedbackForm.css";

function FeedbackForm() {
  // --- Состояние значений полей ---
  // Каждый инпут — управляемый компонент: его значение хранится в state.
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [message, setMessage] = useState("");
```

Сборщик Vite заранее строит граф зависимостей и понимает, какие части библиотеки реально используются..

// --- Состояние полей

```
// ----Состояние ошибки валидации----
// Храним текст ошибки для каждого поля отдельно.
  // Если ошибки нет — значение null.
  const [errors, setErrors] = useState({
    name: null,
    email: null,
    message: null,
  });

  // --- Состояние «тронутых» полей ---
  // Поле считается «тронутым» (touched), если пользователь хотя бы раз
  // вышел из него (blur). До этого момента не показываем ошибки,
  // даже если значение невалидно — это снижает раздражение пользователя.
  const [touched, setTouched] = useState({
    name: false,
    email: false,
    message: false,
  });
```

```
// --- Состояние процесса отправки ---
  // Пока идёт «запрос» — блокируем кнопку и показываем индикатор.
  const [isSubmitting, setIsSubmitting] = useState(false);


  // --- Состояние успешной отправки ---
  // После успешной отправки показываем сообщение вместо формы.
  const [submitted, setSubmitted] = useState(false);
```

продолжение

```
return (
  <form className="feedback-form">
   {/* Поле «Имя» */}
   <div className="field">
    <label htmlFor="name" className="field__label">
     Имя
    </label>
    <input    id="name”  type="text”  className="field__input”    value={name}
     onChange={(e) => setName(e.target.value)}
     onBlur={() => setTouched((prev) => ({ ...prev, name: true }))}
    />
    {/* Блок ошибки появляется только если поле тронуто И есть ошибка */}
    {touched.name && errors.name && (
     <span className="field__error">{errors.name}</span>
    )}
   </div>
```

Оператор spread (...) в JavaScript (комментарий)

Оператор spread (...) в JavaScript — это синтаксический сахар, который «распаковывает» элементы массива или свойства объекта в другой контекст.

Разберём, как это работает:

Для объектов (как в handleChange).

В строке ...prev оператор spread копирует все текущие свойства объекта prev в новый объект:

Пример

```
const prev = { name: 'Alex', age: 25, city: 'Moscow' };
const next = {  ...prev,
  age: 26,              // обновляем существующее свойство
  job: 'developer'      // добавляем новое
};
console.log(next);
// Результат: { name: 'Alex', age: 26, city: 'Moscow', job: 'developer' }
```

Почему это критично в React:

- состояние нельзя менять напрямую (мутировать).
- Spread позволяет создать новый объект с нужными изменениями, сохранив при этом все остальные данные.

Именно поэтому в setFormData используют паттерн prev => ({ ...prev, ... }) — он гарантирует, что мы не потеряем остальные поля формы.

Проверка ошибок полей

```
{/* Поле «Email» */}
  <div className="field">
   <label htmlFor="email" className="field__label">
    Email
   </label>
   <input   id="email“   type="email“  className="field__input"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
    onBlur={() => setTouched((prev) => ({ ...prev, email: true }))}
   />
   {touched.email && errors.email && (
    <span className="field__error">{errors.email}</span>
   )}
  </div>
```

```
{/* Поле «Сообщение» */}
  <div className="field">
   <label htmlFor="message" className="field__label">
    Сообщение
   </label>
   <textarea    id="message"
    className="field__input field__input--textarea"
    value={message}
    onChange={(e) => setMessage(e.target.value)}
    onBlur={() => setTouched((prev) => ({ ...prev, message: true }))}
    rows={5}
   />
   {touched.message && errors.message && (
    <span className="field__error">{errors.message}</span>
   )}
  </div>
```

Отправить

```
 <button
       type="submit"
       className="feedback-form__submit"
       disabled={isSubmitting}
      >
       {isSubmitting ? "Отправка..." : "Отправить"}
      </button>
    </form>
  );
}

export default FeedbackForm;
```

Что здесь происходит:

Каждое поле связано с отдельным состоянием через value и onChange.

onBlur помечает поле как «тронутое» — мы используем функциональную форму setTouched, чтобы не затереть остальные поля.

Ошибка показывается только когда пользователь уже взаимодействовал с полем и валидация нашла проблему.

isSubmitting имитирует загрузку и блокирует повторные нажатия.

Шаг 2. Функции валидации

Добавим функции для проверки каждого поля. Поместите их внутри компонента, перед return.

```
// --- Функции валидации ---
 // Имя: минимум 2 символа, только буквы, пробелы и дефис.
 const validateName = (value) => {
  if (!value.trim()) return "Имя обязательно для заполнения";
  if (value.trim().length < 2) return "Минимум 2 символа";
  if (!/^[a-zA-Za-яA-ЯёЁ\s-]+$/.test(value.trim()))
    return "Только буквы, пробелы и дефис";
  return null; // нет ошибки
 };
 // Email: проверка через регулярное выражение.
 const validateEmail = (value) => {
  if (!value.trim()) return "Email обязателен для заполнения";
  // Простая, но достаточно надёжная проверка формата.
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value.trim()))
    return "Некорректный формат email";
  return null;
 };
```

- `!` — логическое «НЕ»: результат совпадения инвертируется. В коде это обычно выглядит как `!regex.test(email)`, то есть «если тест НЕ прошёл — считаем адрес невалидным».
- `/ ... /` — ограничители регулярного выражения (синтаксис JS).
- `^` — начало строки.
- `[^\s@]+` — один или более символов, которые **не** являются пробелом (`\s`) и не являются `@`. Это «локальная часть» адреса (до @).
- `@` — обязательный символ «собака».
- `[^\s@]+` — снова один или более символов, не являющихся пробелом и `@` (доменная часть до точки).
- `\.` — экранированная точка (разделитель домена и зоны).
- `[^\s@]+` — один или более символов после точки (доменная зона).
- `\$` — конец строки.

Итого шаблон требует структуру: `локальная_часть@домен.зона`, где ни в одной части нет пробелов и лишних `@`.

// Сообщение: минимум 10 символов.

```
const validateMessage = (value) => {
   if (!value.trim()) return "Сообщение обязательно для заполнения";
   if (value.trim().length < 10)
     return "Минимум 10 символов";
   return null;
 };

 // Универсальный объект валидаторов — удобно перебирать в цикле.
 const validators = {
   name: validateName,
   email: validateEmail,
   message: validateMessage,
 };
```

Шаг 3. Обработчик изменения полей (live-валидация)

Теперь обновим onChange так, чтобы при вводе сразу проверялось значение, но ошибка показывалась только для «тронутых» полей.

```
// Обработчик изменения поля.
 // Принимает имя поля и новое значение, проверяет и обновляет состояние.
 const handleChange = (field, value) => {
  // Обновляем значение соответствующего поля через switch.
  switch (field) {
   case "name":
    setName(value);
    break;
   case "email":
    setEmail(value);
    break;
   case "message":
    setMessage(value);
    break;
  // no default    }
  // Если поле уже тронуто — сразу валидируем при наборе.
   // Если нет — не показываем ошибки, пока пользователь не уйдёт из поля.
   if (touched[field]) {
    setErrors((prev) => ({       ...prev,       [field]: validators[field](value),     }));    }  };
```

Обновим onChange в JSX, чтобы использовать handleChange:

```
 // Было:
onChange={(e) => setName(e.target.value)}
 // Стало:
onChange={(e) => handleChange("name", e.target.value)}
```

Аналогично для email и message.

Шаг 4. Обработчик потери фокуса (blur)

При выходе из поля помечаем его как тронутое и проводим валидацию.

```
// Обработчик потери фокуса.
  const handleBlur = (field) => {
    // Помечаем поле как тронутое.
    setTouched((prev) => ({ ...prev, [field]: true
}));


    // Получаем текущее значение поля.
    const currentValue =
      field === "name" ? name : field === "email"
? email : message;
```

```
// Валидируем и записываем результат.
    setErrors((prev) => ({
      ...prev,
      [field]: validators[field](currentValue),
    }));
  };
```

Обновим onBlur в JSX:

```
// Было:
onBlur={() => setTouched((prev) => ({ ...prev, name: true
}))}

// Стало:
onBlur={() => handleBlur("name")}
```

Аналогично для email и message.

Шаг 5. Обработчик отправки формы

```
// Обработчик отправки формы.
  const handleSubmit = (e) => {
    e.preventDefault(); // Предотвращаем перезагрузку страницы.
    // 1. Валидируем все поля сразу.
    const newErrors = {
      name: validateName(name),
      email: validateEmail(email),
      message: validateMessage(message),
    };
    // 2. Записываем ошибки и помечаем все поля как тронутые.
    setErrors(newErrors);
    setTouched({ name: true, email: true, message: true });
    // 3. Если хотя бы одна ошибка есть — не отправляем.
    if (Object.values(newErrors).some((err) => err !== null)) {
return; // Прерываем отправку.    }

    // 4. Имитируем отправку на сервер.
    setIsSubmitting(true);
    // setTimeout имитирует сетевой запрос с задержкой 1.5 секунды.
    setTimeout(() => {
      setIsSubmitting(false);
      setSubmitted(true);
      // В реальном проекте здесь был бы fetch / axios:
      // fetch("/api/feedback", { method: "POST", body: ... })
    }, 1500);
  };
```

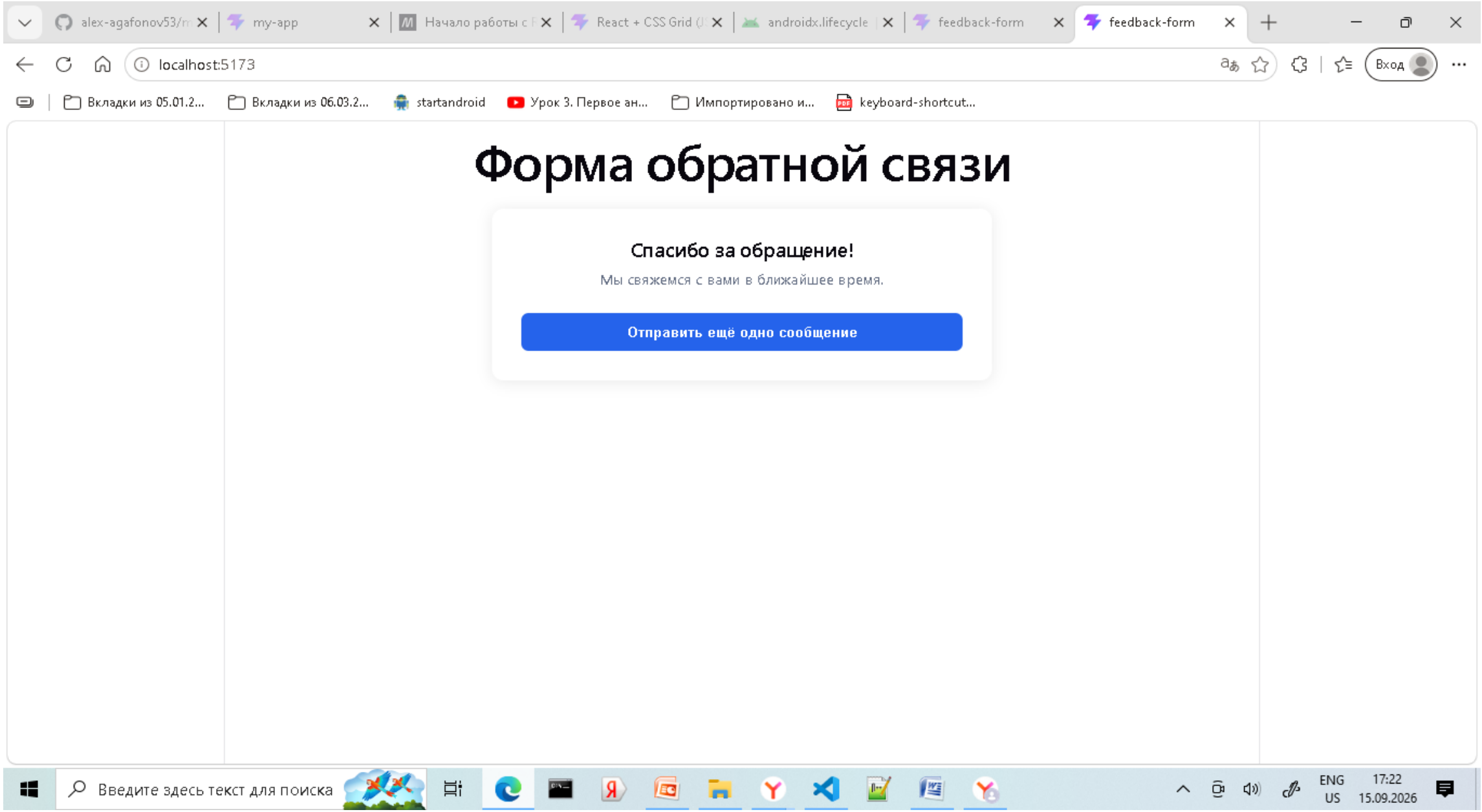
Добавим onSubmit в тег <form>:

<form className="feedback-form" onSubmit={handleSubmit}>

Шаг 6. Условный рендеринг после отправки

Добавим в конец компонента, перед закрывающим </> или return, логику показа успешного экрана:

```
// Если форма успешно отправлена — показываем благодарность.
  if (submitted) {
    return (
      <div className="feedback-form feedback-form--success">
        <h2 className="feedback-form__title">Спасибо за обращение!</h2>
        <p className="feedback-form__text">
          Мы свяжемся с вами в ближайшее время.
        </p>
        <button    type="button“
className="feedback-form__submit"
onClick={() => {
          // Сбрасываем форму к начальному состоянию.
          setName("");
          setEmail("");
          setMessage("");
          setErrors({ name: null, email: null, message: null });
          setTouched({ name: false, email: false, message: false
});
          setSubmitted(false);
        }}
      >
        Отправить ещё одно сообщение
      </button>
    </div>
  );
 }
```


alex-agafonov53/m
my-app
Начало работы с F
React + CSS Grid (
androidx.lifecycle
feedback-form
feedback-form
localhost:5173
Вход
Вкладки из 05.01.2...
Вкладки из 06.03.2...
startandroid
Урок 3. Первое ан...
Импортировано и...
keyboard-shortcut...
Форма обратной связи
Спасибо за обращение!
Мы свяжемся с вами в ближайшее время.
Отправить ещё одно сообщение
Введите здесь текст для поиска
ENG
US
17:22
15.09.2026

Шаг 7. CSS-стилизация состояний полей

Создайте файл src/components/FeedbackForm.css.

Здесь ключевой момент — четыре визуальных состояния для каждого поля.

```
/* src/components/FeedbackForm.css */

/* --- Каркас формы --- */
.feedback-form {
  max-width: 480px;
  margin: 40px auto;
  padding: 32px;
  background: #ffffff;
  border-radius: 12px;
  box-shadow: 0 2px 16px rgba(0, 0, 0, 0.08);
  font-family: system-ui, -apple-system, sans-serif;
}
```

CSS-стилизация состояний полей

```
/* --- Поле --- */
.field {
  margin-bottom: 20px;
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.field__label {
  font-size: 14px;
  font-weight: 600;
  color: #333;
}
```

```
/* --- Базовое состояние инпута (покой) --- */
.field__input {
  width: 100%;
  padding: 10px 14px;
  font-size: 15px;
  border: 2px solid #d0d5dd; /* нейтральная рамка */
  border-radius: 8px;
  background: #f9fafb;
  color: #1d2939;
  outline: none;
        /* убираем стандартный outline,
           кастомизируем через box-shadow */
  transition: border-color 0.2s, box-shadow 0.2s, background 0.2s;
}
```

```
/* --- Состояние фокуса --- */
.field__input:focus {
  border-color: #2563eb;     /* синяя рамка */
  background: #ffffff;
  box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.15); /* мягкое свечение */
}
```

CSS-стилизация состояний полей

```
/* --- Состояние ошибки --- */
/* Класс добавляется через JSX, когда touched[field] && errors[field] */
.field__input--error {
  border-color: #e5484d;
   /* красная рамка */
  background: #fef2f2;
}
.field__input--error:focus {
  border-color: #e5484d;
  box-shadow: 0 0 0 3px rgba(229, 72, 77, 0.15);
}
```

```
/* --- Состояние успеха --- */
/* Класс добавляется, когда поле тронуто и ошибки нет */
.field__input--valid {
  border-color: #30a46c;
/* зелёная рамка */
  background: #f0fdf4;
}
.field__input--valid:focus {
  border-color: #30a46c;
  box-shadow: 0 0 0 3px rgba(48, 164, 108, 0.15);
}
```

```
/* --- Текст ошибки --- */
.field__error {
  font-size: 13px;
  color: #e5484d;
  margin-top: 2px;
}
/* --- Textarea --- */
.field__input--textarea {
  resize: vertical;
  min-height: 100px;
}
```

CSS

```
/* --- Кнопка отправки --- */
.feedback-form__submit {
  width: 100%;
  padding: 12px;
  font-size: 15px;
  font-weight: 600;
  color: #ffffff;
  background: #2563eb;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  transition: background 0.2s;
}
.feedback-form__submit:hover:not(:disabled) {
  background: #1d4ed8;
}
.feedback-form__submit:disabled {
  background: #94a3b8;
  cursor: not-allowed;
}
/* --- Экран успеха --- */
.feedback-form--success {
  text-align: center;
}
.feedback-form__title {
  font-size: 22px;
  margin-bottom: 8px;
}
.feedback-form__text {
  font-size: 15px;
  color: #667085;
  margin-bottom: 24px;
}
```

Шаг 8. Динамические классы в JSX

Чтобы поля получали нужный CSS-класс в зависимости от состояния, добавим вспомогательную функцию внутри компонента (перед return):

jsx

```
  // Возвращает строку классов для инпута в зависимости от состояния.
  const getInputClass = (field) => {
    // Базовый класс
    const classes = ["field__input"];
    // Если поле тронуто и есть ошибка — добавляем класс ошибки.
    if (touched[field] && errors[field]) {
      classes.push("field__input--error");
    }
```

Динамические классы в JSX

```
 // Если поле тронуто и ошибки нет — добавляем класс успеха.
    else if (touched[field] && !errors[field]) {
      classes.push("field__input--valid");
    }
    // Для textarea добавляем отдельный класс.
    if (field === "message") {
      classes.push("field__input--textarea");
    }
  return classes.join(" ");
  };
```

Теперь обновим className у инпутов:

jsx

```
// Было:
className="field__input"

// Стало:
className={getInputClass("name")}
```

Аналогично для email и message.

Шаг 9. Подключение формы в App

Обновите src/App.jsx:

jsx

```
import FeedbackForm from "./components/FeedbackForm";
function App() {
  return (
    <div className="app">
      <h1 style={{ textAlign: "center", fontFamily: "system-ui" }}>
        Форма обратной связи
      </h1>
      <FeedbackForm />
    </div>
  );
}
export default App;
```

Задания для самостоятельной работы

Задание 1. Счётчик символов (базовое)

Добавьте под полем «Сообщение» счётчик вида «число символов» / 500. Ограничьте максимальную длину сообщения 500 символами. Валидация должна учитывать верхний лимит.

Подсказка: используйте maxLength на <textarea> и отдельное состояние для отображения счётчика либо вычисляйте длину из message.length.

Задание 2. Поле «Телефон» (среднее)

Добавьте поле для номера телефона. Валидация: формат +7 (XXX) XXX-XX-XX. Реализуйте маску ввода — при наборе цифр автоматически подставляйте скобки, пробелы и дефисы.

Подсказка: в handleChange форматируйте значение, удаляя все не-цифры, а затем вставляя разделители в нужные позиции.

Задание 3. Зависимая валидация (продвинутое)

Добавьте переключатель «Хочу получить ответ» (checkbox). Если он включён — поле Email становится обязательным; если выключено — Email можно оставить пустым. Реализуйте динамическое обновление правил валидации.

Контрольные вопросы

Чем управляемый компонент отличается от неуправляемого? В каких случаях useRef предпочтительнее useState для поля формы?

Зачем нужно состояние touched? Что изменится, если убрать его и показывать ошибки сразу при вводе?

Почему в setErrors и setTouched используется функциональная форма prev => ...? В каких случаях можно обойтись прямым значением?

Как работает e.preventDefault() в обработчике onSubmit? Что произойдёт без него?

Какие состояния полей реализованы в CSS и как они визуально различаются? Предложите альтернативные визуальные подсказки (иконки, анимации).